1. RecyclerView y el patrón Adapter (Listas eficientes):

Enlace: https://developer.android.com/guide/topics/ui/layout/recyclerview

Explicación: Aprendí que, a diferencia de hacer un simple ciclo o usar listas básicas, el RecyclerView es el estándar en Android para mostrar colecciones de datos de forma eficiente. Su principal ventaja es que "recicla" las tarjetas (vistas) que salen de la pantalla al hacer scroll, ahorrando memoria. Para implementarlo, descubrí que es obligatorio crear un Adapter (que inyecta los datos) y un ViewHolder (que guarda las referencias directas a los elementos de la interfaz, como los TextView o ImageView, para no tener que buscarlos constantemente en cada recarga).

2. SharedPreferences (Almacenamiento local persistente):

Enlace: https://developer.android.com/training/data-storage/shared-preferences

Explicación: Para mantener el estado de las actividades (si están marcadas como "Completadas" o "Pendientes") incluso si la app se cierra, investigué el uso de SharedPreferences. Este componente permite guardar datos ligeros en formato clave-valor (como booleanos o textos). Aprendí que para leer un dato basta con pedirlo por su nombre (clave), pero para guardar un cambio es necesario abrir un Editor, hacer la modificación (ej. putBoolean()) y usar el método .apply() para que el guardado se ejecute en segundo plano sin congelar la pantalla.

3. SensorManager y SensorEventListener (Acceso al Podómetro):

Enlace: https://developer.android.com/guide/topics/sensors/sensors_motion

Explicación: Para lograr que la aplicación cuente los pasos físicos del usuario, tuve que investigar cómo Android maneja el hardware nativo. Descubrí que se requiere instanciar el servicio SensorManager y acceder específicamente al sensor TYPE_STEP_COUNTER. Además, aprendí que la clase debe implementar una interfaz llamada SensorEventListener, la cual me obliga a sobreescribir el método onSensorChanged(), que es el que reacciona en tiempo real cada vez que el chip del teléfono detecta un nuevo paso. También noté que es estrictamente necesario pedir permisos de ACTIVITY_RECOGNITION al usuario.

4. Intents (Navegación entre pantallas):

Enlace: https://developer.android.com/guide/components/intents-filters

Explicación: Aprendí que en el desarrollo nativo de Android no se usa un "Navigator" para cambiar de vistas, sino que el sistema funciona a través de Intents (Intenciones). Para ir de la pantalla principal a la de detalles o al perfil, instancié un Intent explícito, indicándole el contexto actual y la clase de destino (ej. Detail::class.java). Además, investigué el método putExtra(), el cual actúa como un "paquete de envío" que me permitió mandar datos específicos (como el título de la actividad o la cantidad de pasos) para que la segunda pantalla los reciba y los muestre.